

Course: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Testing application

1. Testing Methodology

To accurately measure the performance of our workload, we executed an automated benchmark script. To make sure project requirements are met, the testing environment was strictly controlled:

- **Cold Starts:** The database memory buffers (**BUFFER_CACHE** and **SHARED_POOL**) were flushed before every single run to prevent the database from artificially speeding up queries using cached RAM.
- **State Restoration:** A **ROLLBACK** command was executed at the end of each run to instantly undo all **INSERT**, **UPDATE**, and **DELETE** operations, ensuring the database state was completely identical before each execution.
- **Logging:** All execution times and query plans were automatically spooled and saved to a local benchmark file.

2. Execution Times Table

Note: All times are recorded in seconds.

Transaction	Run 1	Run 2	Run 3	Run 4	Run 5	Min Time	Max Time	Avg Time
Q1	1.557	1.421	1.405	1.416	1.408	1.405	1.557	1.441
Q2	1.046	1.017	1.004	1.001	1.005	1.004	1.046	1.015
Q3	0.579	0.576	0.570	0.563	0.569	0.563	0.579	0.571
Q4 (INSERT)	0.238	0.206	0.202	0.205	0.204	0.202	0.238	0.211
Q5 (UPDATE)	5.822	5.818	5.861	5.82	5.861	5.861	5.818	5.836
Q6 (DELETE)	1.112	0.758	1.259	1.699	1.310	0.758	1.699	1.228
Whole Workload	10.354	9.796	10.301	10.704	10.357	9.796	10.704	10.302

3. Conclusions

The automated benchmark successfully captured the performance of the workload over 5 independent runs.

The buffer flushing methodology proved effective, as the execution times remain consistent across all runs with relatively low variance. The total workload execution time averages approximately 10.3 seconds, with results ranging from 9.796 s to 10.704 s, indicating stable and repeatable performance.

Most of the queries (Q1–Q4) execute relatively quickly, generally below 1.5 seconds, with Q3 and Q4 being particularly efficient (well below 1 second). However, the workload is heavily dominated by Query 5 (UPDATE), which averages 5.836 seconds, making it the primary performance bottleneck.

Additionally, Q6 (DELETE) shows noticeable variability between runs (0.758 s to 1.699 s), suggesting potential sensitivity to data distribution or execution conditions.

